

Public-Key Cryptography

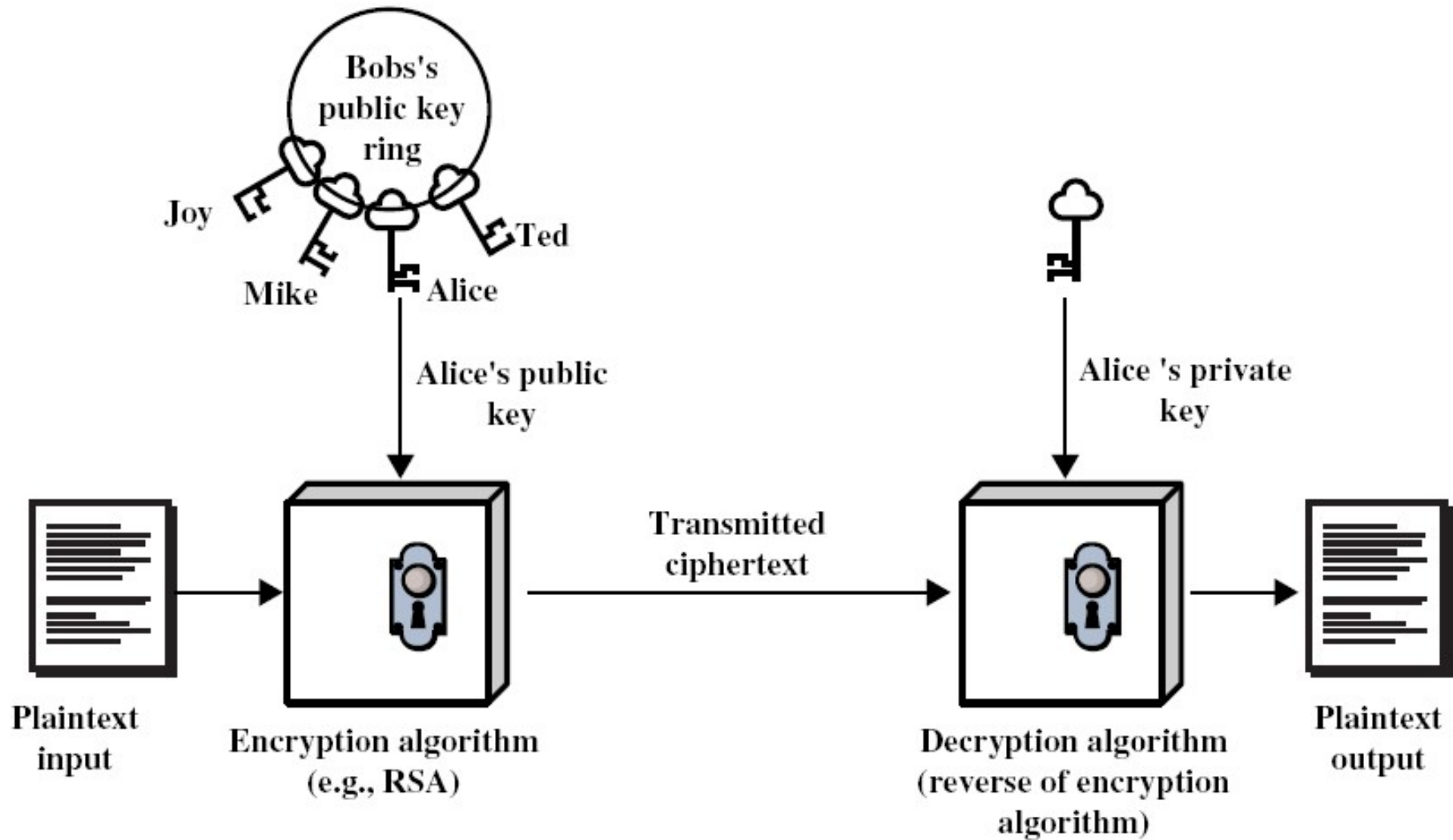
Why Public-Key Cryptography?

- developed to address two key issues:
 - **key distribution** - how to have secure communications in general without having to trust a KDC with your key
 - **digital signatures** - how to verify a message comes intact from the claimed sender
- Public key schemes, and 1st practical scheme, which was for **key distribution only, was published in 1976 by Whitfield Diffie & Martin Hellman** at Stanford Uni
 - known earlier in classified community

Public-Key Cryptography Principles

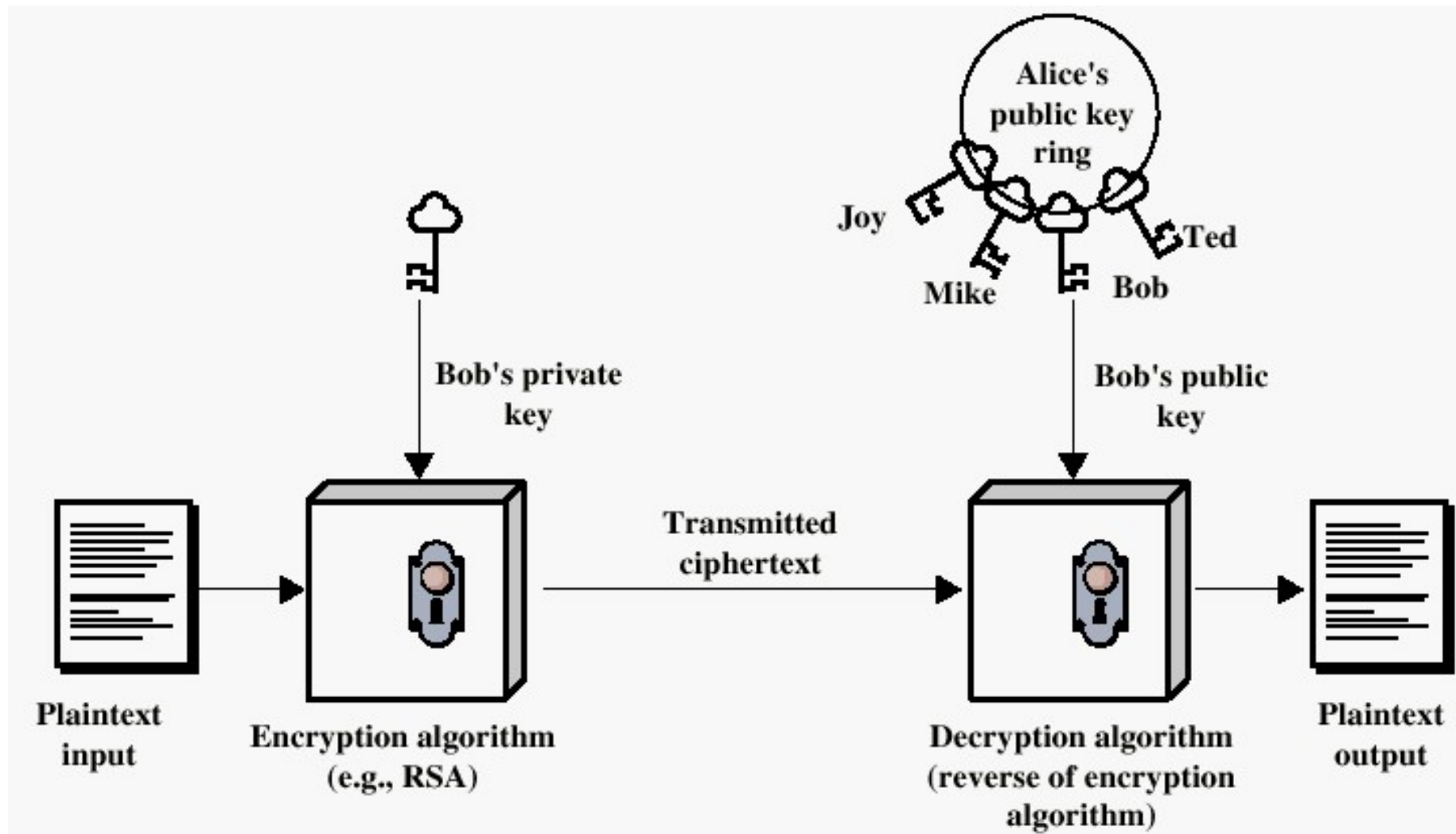
- The use of two keys has consequences in: key distribution, confidentiality and authentication.
- The scheme has six ingredients (see Figure (a))
 - Plaintext
 - Encryption algorithm
 - Public and private key
 - Ciphertext
 - Decryption algorithm

Encryption using Public-Key system



(a) Encryption

Authentication using Public-Key System



Applications for Public-Key Cryptosystems

- Three categories:
 - **Encryption/decryption:** The sender encrypts a message with the recipient's public key.
 - **Digital signature:** The sender "signs" a message with its private key.
 - **Key exchange:** Two sides cooperate to exchange a session key.

Table 3.2 Applications for Public-Key Cryptosystems

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No
Elliptic Curve	Yes	Yes	Yes

Requirements for Public-Key Cryptography

1. Computationally easy for a party B to generate a pair (public key KU_b , private key KR_b)
2. Easy for sender to generate ciphertext: $C = E_{KU_b}(M)$
3. Easy for the receiver to decrypt ciphertext using private key:

$$M = D_{KR_b}(C) = D_{KR_b}[E_{KU_b}(M)]$$

Requirements for Public-Key Cryptography (contd.)

4. Computationally infeasible to determine private key (KR_b) without knowing public key (KU_b)
5. Computationally infeasible to recover message M , knowing KU_b and ciphertext C
6. Either of the two keys can be used for encryption, with the other used for decryption:

$$M = D_{KRb}[E_{KU_b}(M)] = D_{KU_b}[E_{KRb}(M)]$$

Public-Key Cryptographic Algorithms

- RSA and Diffie-Hellman
- **RSA** - Ron Rivest, Adi Shamir and Len Adleman at MIT, in 1977.
 - RSA is a block cipher
 - The most widely implemented
- **Diffie-Hellman**
 - Exchange a secret key securely
 - Compute discrete logarithms

RSA Algorithm

$$C = M^e \bmod (n=pq)$$

Plaintext: M

Ciphertext: C

$$M = C^d \bmod n$$

Encryption

- Plaintext: $M < n$
- Ciphertext: $C = M^e \bmod n$

Decryption

- Ciphertext: C
- Plaintext: $M = C^d \bmod n$

The RSA Algorithm - Key Generation

1. Select p, q p and q both prime
2. Calculate $n = p \times q$
3. Calculate $\Phi(n) = (p-1)(q-1)$
4. Select integer e $\gcd(\Phi(n), e) = 1; 1 < e < \Phi(n)$
5. Calculate d $d = e^{-1} \bmod \Phi(n)$
6. Public Key $KU = \{e, n\}$
7. Private key $KR = \{d, n\}$

RSA Algorithm Example

1. Choose $p = 3$ and $q = 11$
2. Compute $n = p * q = 3 * 11 = 33$
3. Compute $\varphi(n) = (p - 1) * (q - 1) = 2 * 10 = 20$
4. Choose e such that $1 < e < \varphi(n)$ and e and n are coprime. Let $e = 7$
5. Compute a value for d such that $(d * e) \% \varphi(n) = 1$.
One solution is $d = 3$ $[(3 * 7) \% 20 = 1]$
6. Public key is $(e, n) \Rightarrow (7, 33)$
7. Private key is $(d, n) \Rightarrow (3, 33)$
8. The encryption of $m = 2$ is $c = 2^7 \% 33 = 29$
9. The decryption of $c = 29$ is $m = 29^3 \% 33 = 2$

Class Exercise: Work with $p=17$, $q= 11$

RSA Example - Key Setup

1. Select primes: $p=17$ & $q=11$
2. Calculate $n = pq = 17 \times 11 = 187$
3. Calculate $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$
4. Select e : $\gcd(e, 160) = 1$; **choose** $e=7$
5. Determine d : $de \bmod 160$ and $d < 160$
Value is $d=23$ **since** $23 \times 7 = 161$
and $161 \bmod 160 = 1$

Publish public key $PU = \{7, 187\}$

Keep secret private key $PR = \{23, 187\}$

RSA Example - En/Decryption

➤ sample RSA encryption/decryption is:

➤ given message $M = 88$ (nb. $88 < 187$)

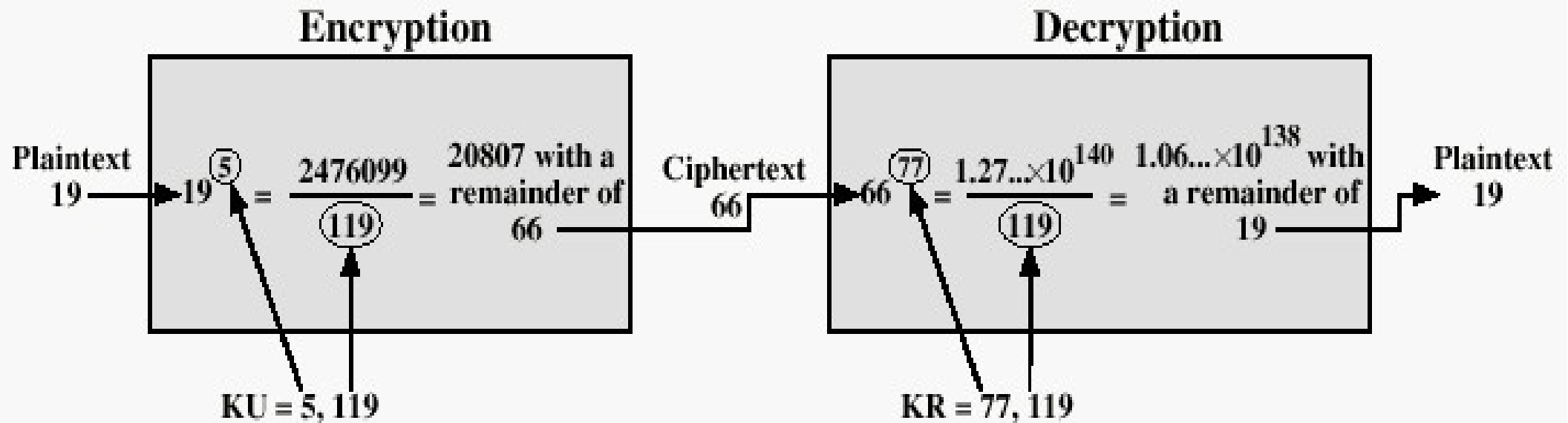
➤ encryption:

$$C = 88^7 \bmod 187 = 11$$

➤ decryption:

$$M = 11^{23} \bmod 187 = 88$$

2nd Example of RSA Algorithm



What are p
and q values
in this case?

Figure 3.9 Example of RSA Algorithm

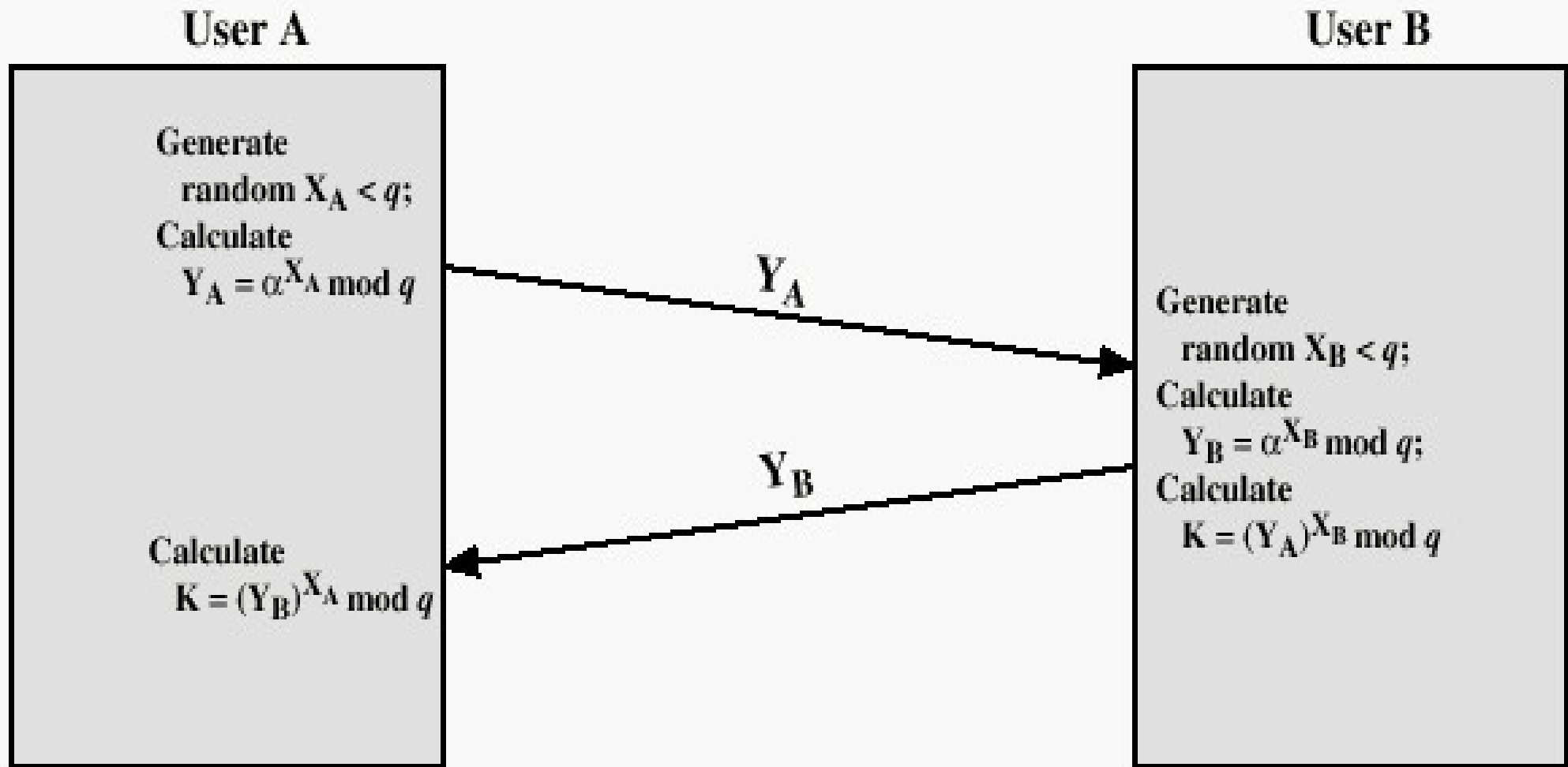
Diffie-Hellman Key Exchange

- a public-key distribution scheme
 - cannot be used to exchange an arbitrary message
 - rather it can establish a common key
 - known only to the two participants
- value of key depends on the participants (and their private and public key information)
- based on exponentiation in a finite (Galois) field (modulo a prime or a polynomial) - easy
- security relies on the difficulty of computing discrete logarithms (similar to factoring) - hard

Diffie-Hellman Setup

- all users agree on global parameters:
 - large prime integer or polynomial q
 - a a primitive root mod q
- each user (eg. A) generates their key
 - chooses a secret key (number): $x_A < q$
 - compute their **public key**: $y_A = a^{x_A} \bmod q$
- each user makes public that key y_A

Diffie-Hellman Key Exchange



Primitive Root of a Prime Number - Example

Elementary example [\[edit \]](#)

The number 3 is a primitive root modulo 7^[1] because

$$\begin{array}{lclclclclclclcl} 3^1 & = & 3 & = & 3^0 \times 3 & \equiv & 1 \times 3 & = & 3 & \equiv & 3 \pmod{7} \\ 3^2 & = & 9 & = & 3^1 \times 3 & \equiv & 3 \times 3 & = & 9 & \equiv & 2 \pmod{7} \\ 3^3 & = & 27 & = & 3^2 \times 3 & \equiv & 2 \times 3 & = & 6 & \equiv & 6 \pmod{7} \\ 3^4 & = & 81 & = & 3^3 \times 3 & \equiv & 6 \times 3 & = & 18 & \equiv & 4 \pmod{7} \\ 3^5 & = & 243 & = & 3^4 \times 3 & \equiv & 4 \times 3 & = & 12 & \equiv & 5 \pmod{7} \\ 3^6 & = & 729 & = & 3^5 \times 3 & \equiv & 5 \times 3 & = & 15 & \equiv & 1 \pmod{7} \end{array}$$

Here we see that the period of 3^k modulo 7 is 6. The remainders in the period, which are 3, 2, 6, 4, 5, 1, form a rearrangement of all nonzero remainders modulo 7, implying that 3 is indeed a primitive root modulo 7. Curiously, permutations created in this way (and their circular shifts) have been shown to be [Costas arrays](#).

Diffie-Hellman Key Exchange

- shared session key for users A & B is K_{AB} :

$$\begin{aligned} K_{AB} &= a^{x_A \cdot x_B} \bmod q \\ &= y_A^{x_B} \bmod q \quad (\text{which } \mathbf{B} \text{ can compute}) \\ &= y_B^{x_A} \bmod q \quad (\text{which } \mathbf{A} \text{ can compute}) \end{aligned}$$

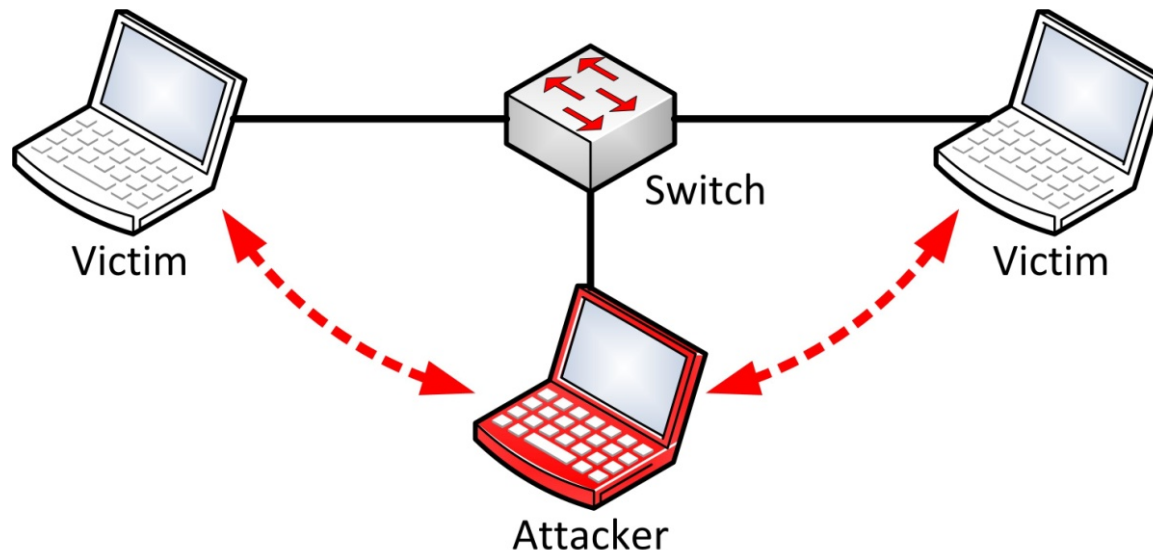
- K_{AB} is used as session key in private-key encryption scheme between Alice and Bob
- if Alice and Bob subsequently communicate, they will have the **same** key as before, unless they choose new public-keys
- attacker needs an x , must solve discrete log

Diffie-Hellman Example

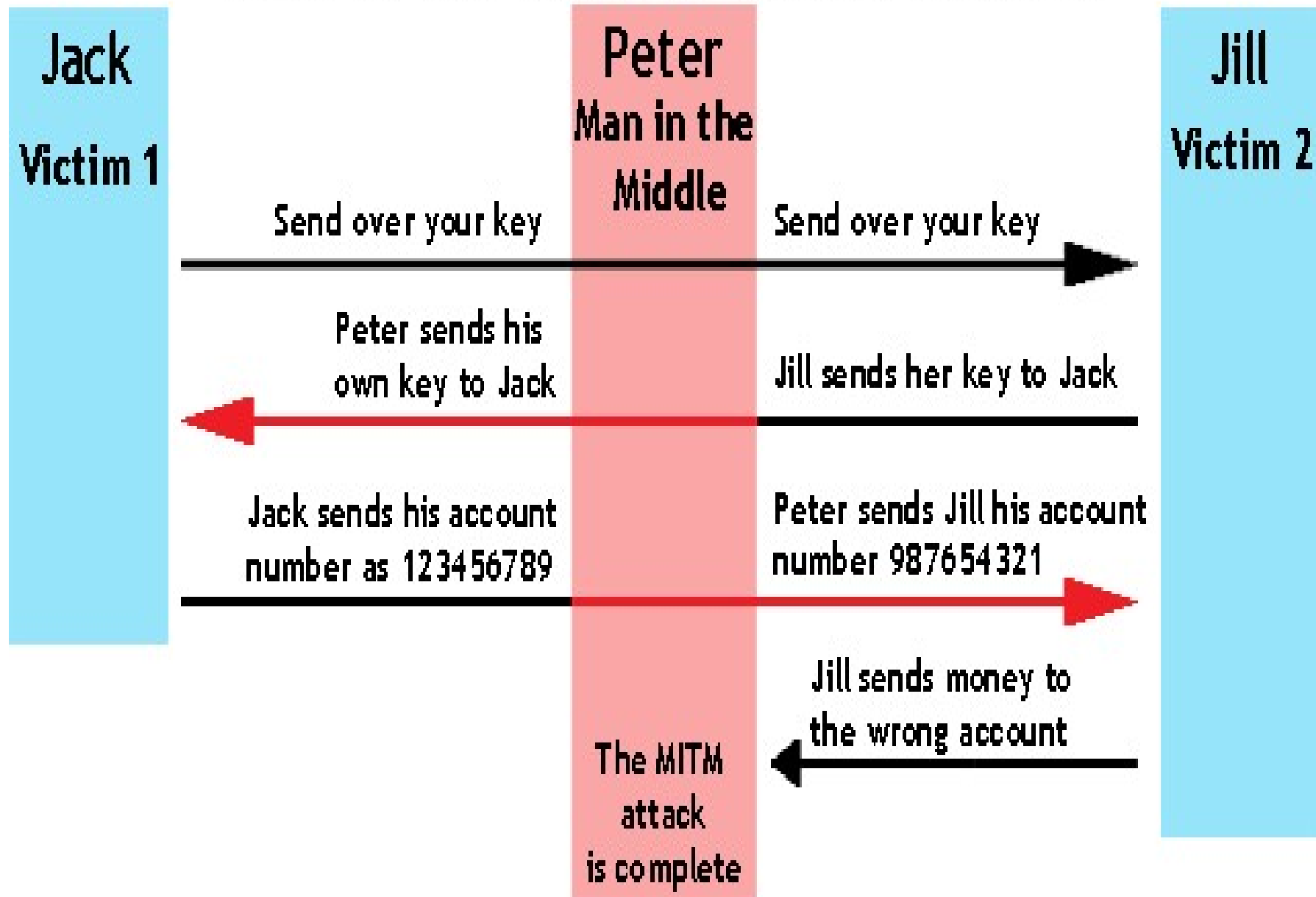
- users Alice & Bob who wish to swap keys:
- agree on prime $q=353$ and $\alpha=3$
- select random secret keys:
 - A chooses $x_A=97$, B chooses $x_B=233$
- compute public keys:
 - $y_A=3^{97} \bmod 353 = 40$ (Alice)
 - $y_B=3^{233} \bmod 353 = 248$ (Bob)
- compute shared session key as:
 - $K_{AB} = y_B^{x_A} \bmod 353 = 248^{97} = 160$ (Alice)
 - $K_{AB} = y_A^{x_B} \bmod 353 = 40^{233} = 160$ (Bob)

Diffie-Hellman Key Exchange Disadvantage

- It suffers from the **Man in the Middle Attack**.
- an **attack** where the attacker secretly relays and possibly alters the communication between two parties who believe they are directly communicating with each other.



Man-in-the-Middle Attack Example



Other Public-Key Cryptographic Algorithms

- Digital Signature Standard (DSS)
 - Makes use of the SHA-1
 - Not for encryption or key exchange
- Elliptic-Curve Cryptography (ECC)
 - Good for smaller bit size
 - Low confidence level, compared with RSA
 - Very complex

Key Management

- public-key encryption helps address key distribution problems
- have two aspects of this:
 1. distribution of public keys
 2. use of public-key encryption to distribute secret keys

Distribution of Public Keys

- can be considered as using one of:
 1. public announcement
 2. publicly available directory
 3. public-key authority
 4. public-key certificates

1. Public Announcement

- users distribute public keys to recipients or broadcast to community at large
 - eg. Append keys to email messages or post to news groups or email list
- major weakness is forgery
 - anyone can create a key claiming to be someone else and broadcast it
 - until forgery is discovered can masquerade as claimed user

Example

And here's the complete and *visible* PGP PUBLIC KEY BLOCK:

```
<p>
The information below may be used to verify my signatures and encrypt communications to me.
<p>
<p>
-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: GnuPG v1.4.6 (GNU/Linux)
<p>
mQGIBegIJGYRBAC54vZVXjK5l4VRSiUC6XGMgEOjEFgWvruVr/PXBk0hbnZ47D8j
...
-----END PGP PUBLIC KEY BLOCK-----</span></div>

<div style="text-align:center;">
```

2. Publicly Available Directory

- can obtain greater security by registering keys with a public directory
- directory must be trusted with properties:
 - contains {name,public-key} entries
 - participants register securely with directory
 - participants can replace key at any time
 - directory is periodically published
 - directory can be accessed electronically
- still vulnerable to tampering or forgery

Example



```
-----BEGIN PGP PUBLIC KEY BLOCK-----  
Version: GnuPG v1.4.11 (GNU/Linux)
```

```
mQENBFAH2BIBCAC43yy6LeB90z30/6VirqVozfi2LROUhdOxhsPgNmoiHQEvo8rq  
SntC1ycUSFnDdkcLYZQ49JkLzLuzc02YkL9amfCLvijAHO7HEeluGCsjsxAS0etw  
LSiG72VuBPGeVnZ0ItfuN+2zTOtcee6aE4/xoijNqIEScoyLKdgIY/tDg/SbjRTg  
Nse4UByP8dz8cFO8KLgXOwb7dtzArYXMMhPIWZHsF51Uvg/cNGU78v/sco2Q0Qz0  
Ohm7mEmQF0//knPmtP+wa9qBZCr4oAKJiMqBU98cLdtu7u3+vbTAPTxxJA4gSuQF  
XqS7U/Ic9ES1Ykp5/CPnYgNK71Pby3OR/AidABEBAAAG0PGVuY3J5cHRlZCB1bm91  
bWEgKG5vIHh0b3N3b3JkKSA8ZW5jcnlwdGVkZW5pZ21hQGFrZG12aXgub3JnPokB  
PgQTAQIAKAUCUAFMEgIbIwUJCWYBgAYLCQgHAWIGFQgCCQoLBBYCAwECHgECF4AA  
CgkQnOEZY78xntTQsiggAiMdgqT/cofMhhKAot2E7kdI5moGKBaiAOqhe1jvdT2nu  
NlTEUArmKfRUNGdyUe09i6NcjyZYitzaFA5ofvzQRXvzU28t99uXhfd2J1tUX0PI  
gO5OnZgErNtNs83kdrjOdXes4vZWjhkiACe0OQMqWVx1DcVdj1VmNWSfMzLZAcJy  
bnbfBGyRJJYOei1fqvr+u4E3NoDGSHGNAMd1B7PROzwzjeleEIW38byGuyggu7jym  
HvwJeWnyzeZE26ceOnVIGgw26DyX1fCTCCTZEukswdUwmkRK1Y+PltU60r//QY2D  
syH5HzGASxt/sBi0geODFGzhSFscfkkazRbMqYdk6bkBDQRQB8wSAQgA8vX9Eyfo  
If6HOUNNx8Yo/iTs/1RYeCAV1dhrmvuW80anIv4eV5IKdb0oNRp6Pn6HW9/AW+84  
J39I12OvQC98Y8mrKbDuabgaTsTx27FLtY3xqFvKahYTsaIRfSN5MC2hWCsx2OQu  
ezLUg79U8qL5kf1QAK1HUSRVftdNO7DhI+VFjVsNzHaxUSpC3xIXow0OojONxxKp  
fyG4Vvmk3CKRPFpUx13qaDS1RGPxRfi+5oBDDR9esNjUg1tP0+tBdeHa+qqP84Gw  
C6rFa/p6jbxjGLTA9cpnv0+SQi4Rxt12MLTyL1vsFgZhoBB5U9GceYTEEW1IKNJb  
j6K4frz+ZQO3qWARAQAB1QE1BBgBAGAPBQJQB8wSAhsMBQkJZgGAAAOJEJzhGWO/  
MZ00ncEH+QHWP9PB3Egm9EZ1bjhUVKWo2xSsXCmKYki1W1+MYK8vqjd1D0H+pxAu  
M2KDwuWtKGn1jXfWPo2CbnKJtWsmJDAValfUkRIOzDj9nSwwvknftK94gaj8CoIS  
bqyOh5LunkjL+9kWrsk63Ldp9ut+CSnAopn4EHMPpVHLNX7iSPHu1ytwtT6yFuX8  
6IOUA5SPA6ejxNYm55TBF0Nx83EYe1DtB+zpS2SZEf1e1ce2YBRgCQZyAzMqIO+U  
oOYxnJQ3dD/FmPJjOnmoK1MVAQb62fZXX0YxLtshtyU7yNwtF1lk1jW7urOtEsuP  
pHaAeitfUAWDs06zjmZqT6V2ssSUUFc=  
=HjZ+  
-----END PGP PUBLIC KEY BLOCK-----
```

Public-Key Authority

- improve security by tightening control over distribution of keys from directory
- has properties of directory
- and requires users to know **public key for the directory**
- then users interact with directory to obtain any desired public key securely
 - does require real-time access to directory when keys are needed

CAs in India

Licensed CAs

- Safescrypt
- IDRBT
- National Informatics Centre
- TCS
- GNFC
- e Mudhra CA
- CDAC CA
- Capricorn CA
- NSDL e-Gov CA

Licensed CAs



Disclosure Record of
Safescrypt CA



Disclosure Record of iTrust
CA (IDRBT)



National Informatics
Centre



Disclosure Record of
TCS CA



Disclosure Record of (n)Code
Solutions CA



Disclosure Record of
e-Mudhra CA



Disclosure Record of CDAC CA



Disclosure Record of
Capricorn CA

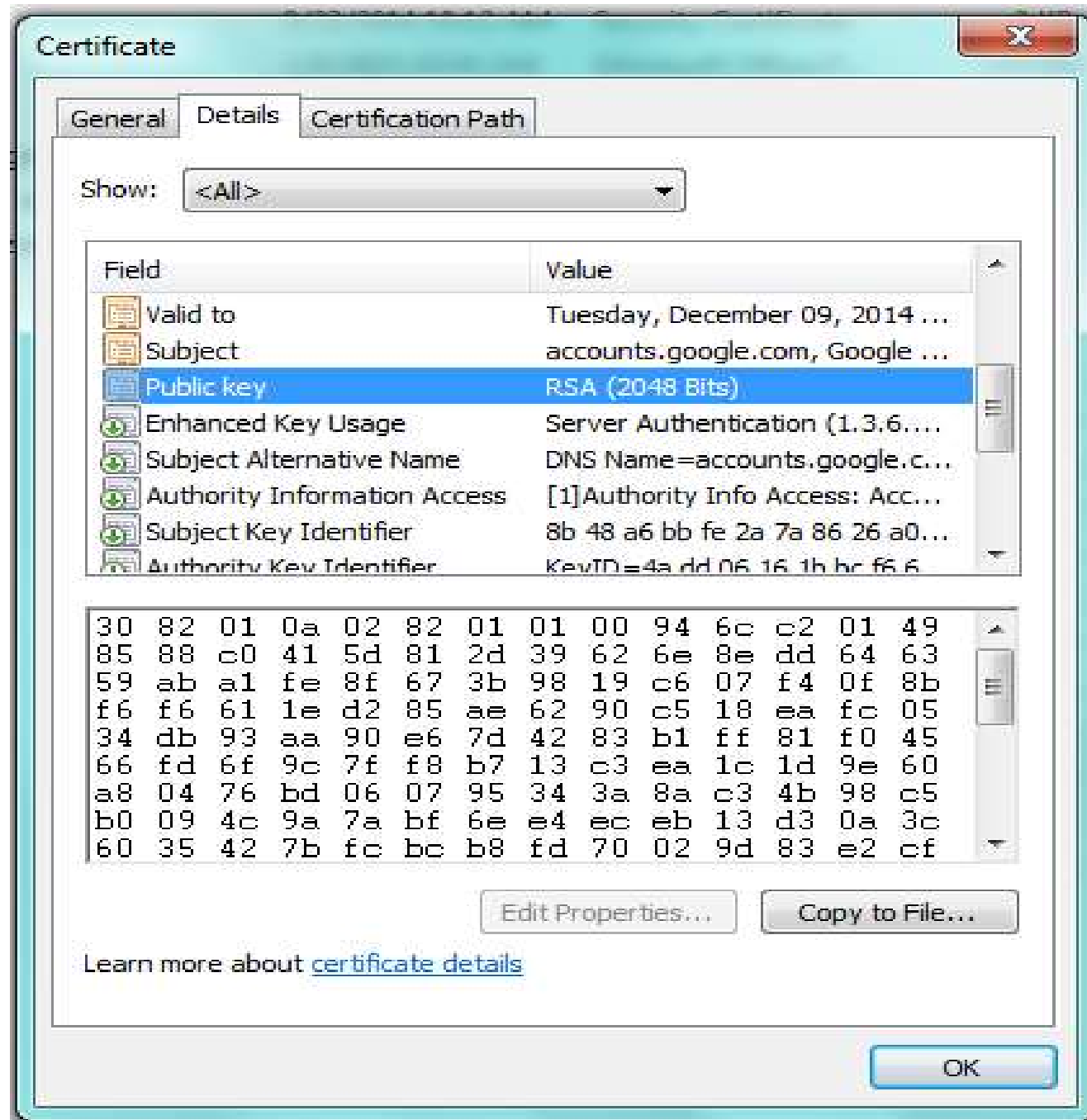


Disclosure Record of NSDL
e-Gov CA

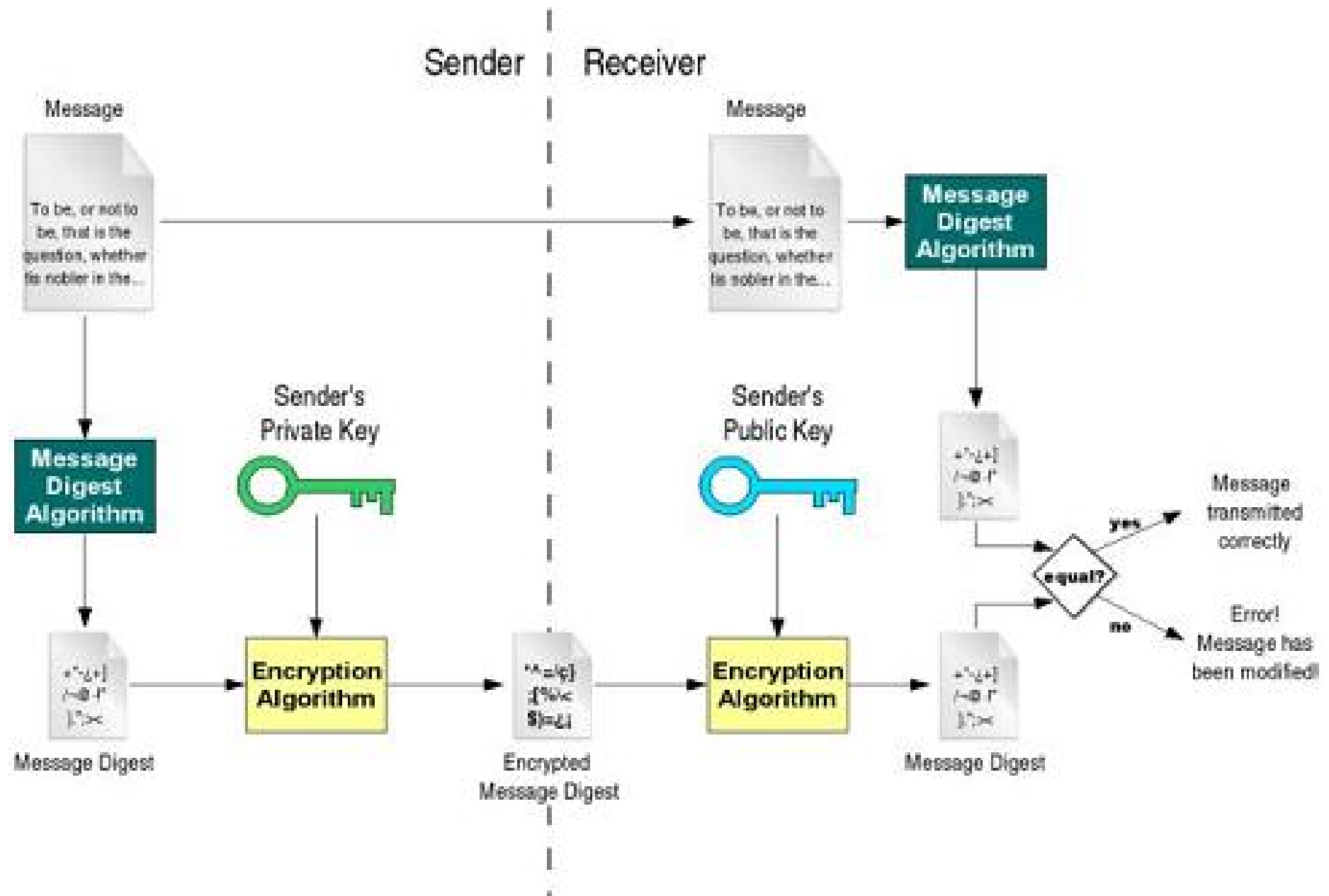
Public-Key Certificates

- certificates allow key exchange without real-time access to public-key authority
- a certificate binds **identity** to **public key**
 - usually with other info such as period of validity, rights of use etc
- with all contents **signed** by a trusted Public-Key or Certificate Authority (CA)
- can be verified by anyone who knows the public-key authorities public-key

Example



Key Management: Public-Key Certificate Use



Public-Key Distribution of Secret Keys

- use previous methods to obtain public-key
- can use for secrecy or authentication
- but public-key algorithms are slow
- so usually want to use private-key encryption to protect message contents
- hence need a session key
- have several alternatives for negotiating a suitable session

Security Concerns

- key concerns are **confidentiality** and **timeliness**
- to provide confidentiality must encrypt identification and session key info
- which requires the use of previously shared private or public keys
- need timeliness to prevent **replay attacks**
- provided by using sequence numbers or timestamps or challenge/response

KERBEROS

What?

Computer network authentication protocol

How?

Works on the basis of 'tickets'

Why?

Allow nodes communicating over a non-secure network to prove their identity to one another in a secure manner.

- Users wish to access services on servers.
- Three threats exist:
 - User pretend to be another user.
 - User alter the network address of a workstation.
 - User eavesdrop on exchanges and use a replay attack.

KERBEROS

- Provides a centralized **Authentication Server** to authenticate users to servers and servers to users.
- Relies on conventional encryption, making no use of public-key encryption
- Two versions: version 4 and 5
- Version 4 makes use of DES

Kerberos Requirements

- its first report identified requirements as:
 - transparent
 - reliable
 - scalable
 - secure
- implemented using an authentication protocol based on Needham-Schroeder

Kerberos Version 4

- Terms:
 - C = Client
 - AS = authentication server
 - V = server
 - ID_c = identifier of user on C
 - ID_v = identifier of V
 - P_c = password of user on C
 - AD_c = network address of C
 - K_v = secret encryption key shared by AS and V
 - TS = timestamp
 - $||$ = concatenation

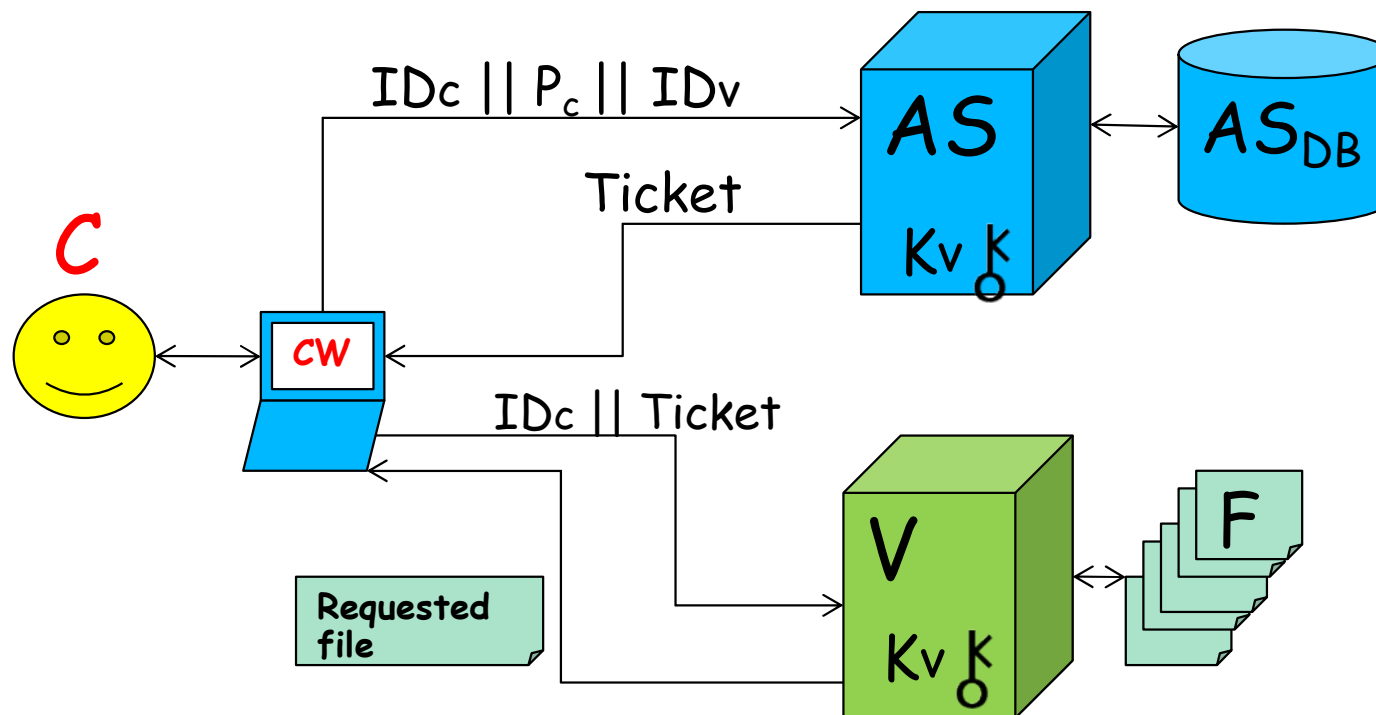
A Simple Authentication Dialogue

(1) $C \rightarrow AS$: $ID_c || P_c || ID_v$

(2) $AS \rightarrow C$: Ticket

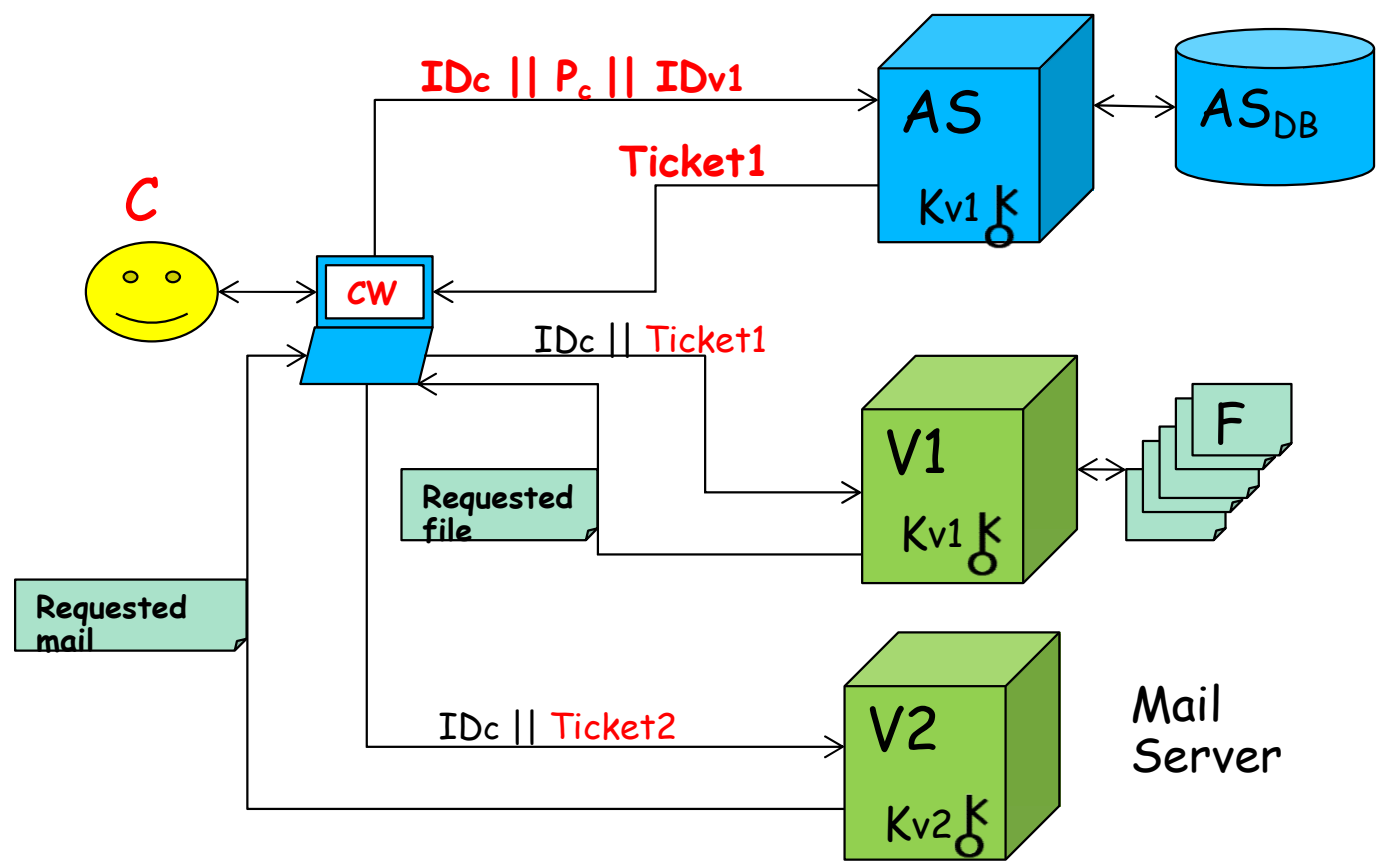
(3) $C \rightarrow V$: $ID_c || Ticket$

$Ticket = E_{K_v}[ID_c || AD_c || ID_v]$



C: Client
CW: Client Workstation
AS: Authentication Server
AS_{DB}: AS database
V: File Server
F: File
Kv: Key known only to AS and V

Disadvantages in the Authentication Dialogue



- Problems:

1. As password is sent as plaintext, hackers may get hold of it and use illegally.
2. For different server, different ticket is to be obtained by the client.

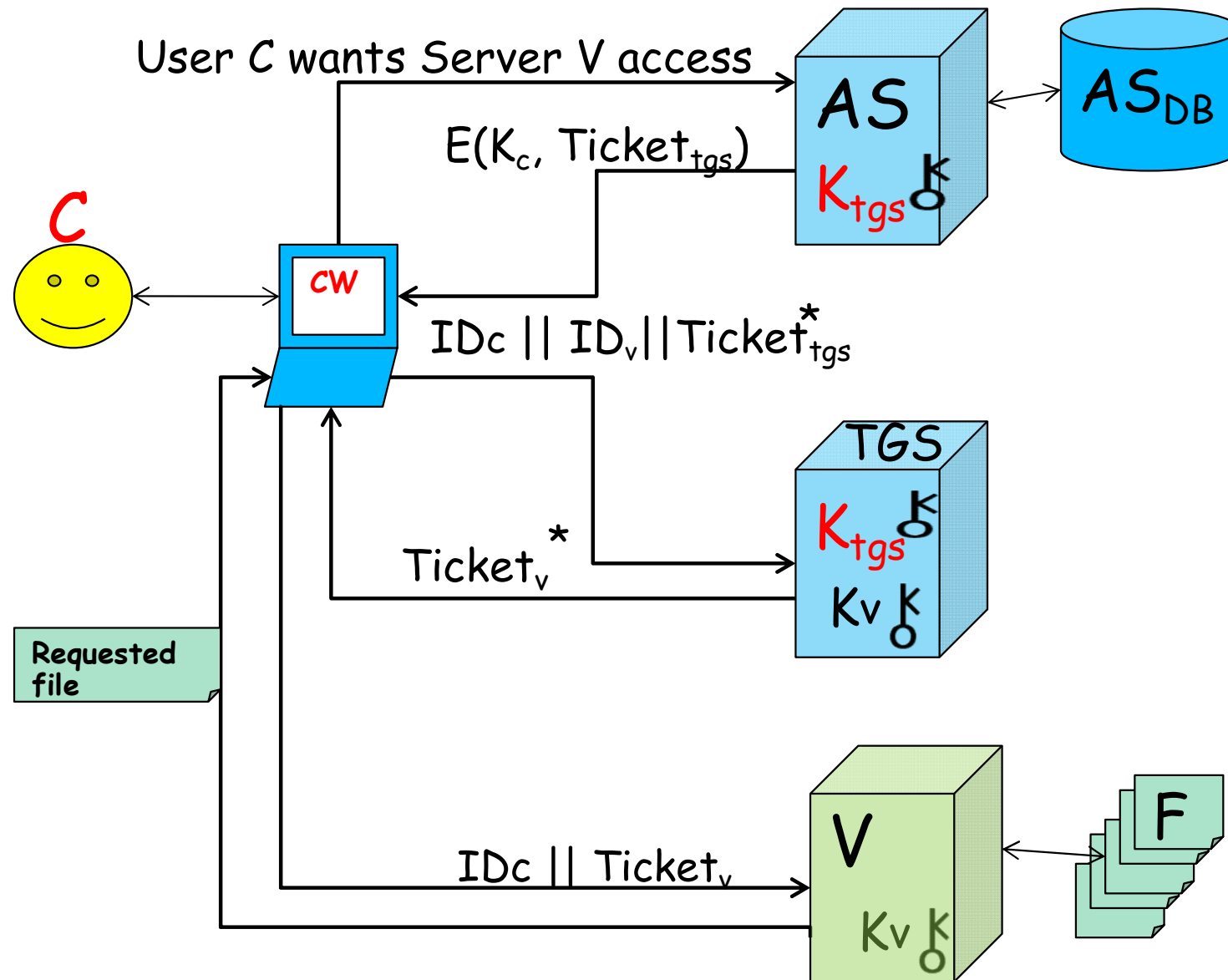
Overcoming Problems in Kerberos v4:

1. Password not to be sent to AS
2. Introduce new sever: Ticket Granting server (TGS)

Overview

- a basic third-party authentication scheme
- have an Authentication Server (AS)
 - users initially negotiate with AS to identify self
 - AS provides a non-corruptible authentication credential (ticket granting ticket TGT)
- have a Ticket Granting server (TGS)
 - users subsequently request access to other services from TGS on basis of users TGT
- using a complex protocol using DES

A more secure Authentication Dialogue



$$K_c = f(P_c)$$

$$\text{Ticket}_{tgs} = E(K_{tgs}, [ID_c || AD_c || ID_{tgs} || TS_1 || \text{Lifetime}_1])$$

$$\text{Ticket}_v = E(K_v, [ID_c || AD_c || ID_v || TS_2 || \text{Lifetime}_2])$$

Problems with Version 4 Authentication Dialogue (mentioned in earlier slide)

- Problems:

1. Lifetime associated with the ticket-granting ticket

- a) If too short → repeatedly asked for password

As password is sent as plaintext, hackers may get hold of it and use illegally.

- b) If too long → greater opportunity to replay

The threat is that an opponent will steal the ticket and use it before it expires

2. If opponent captures $\text{Ticket}_{\text{tgs}}$ and uses it before it expires, the opponent has access to the corresponding service.

Solution to this: So TGS must authenticate itself to the client so that client passes the request to the right TGS and not to false server.

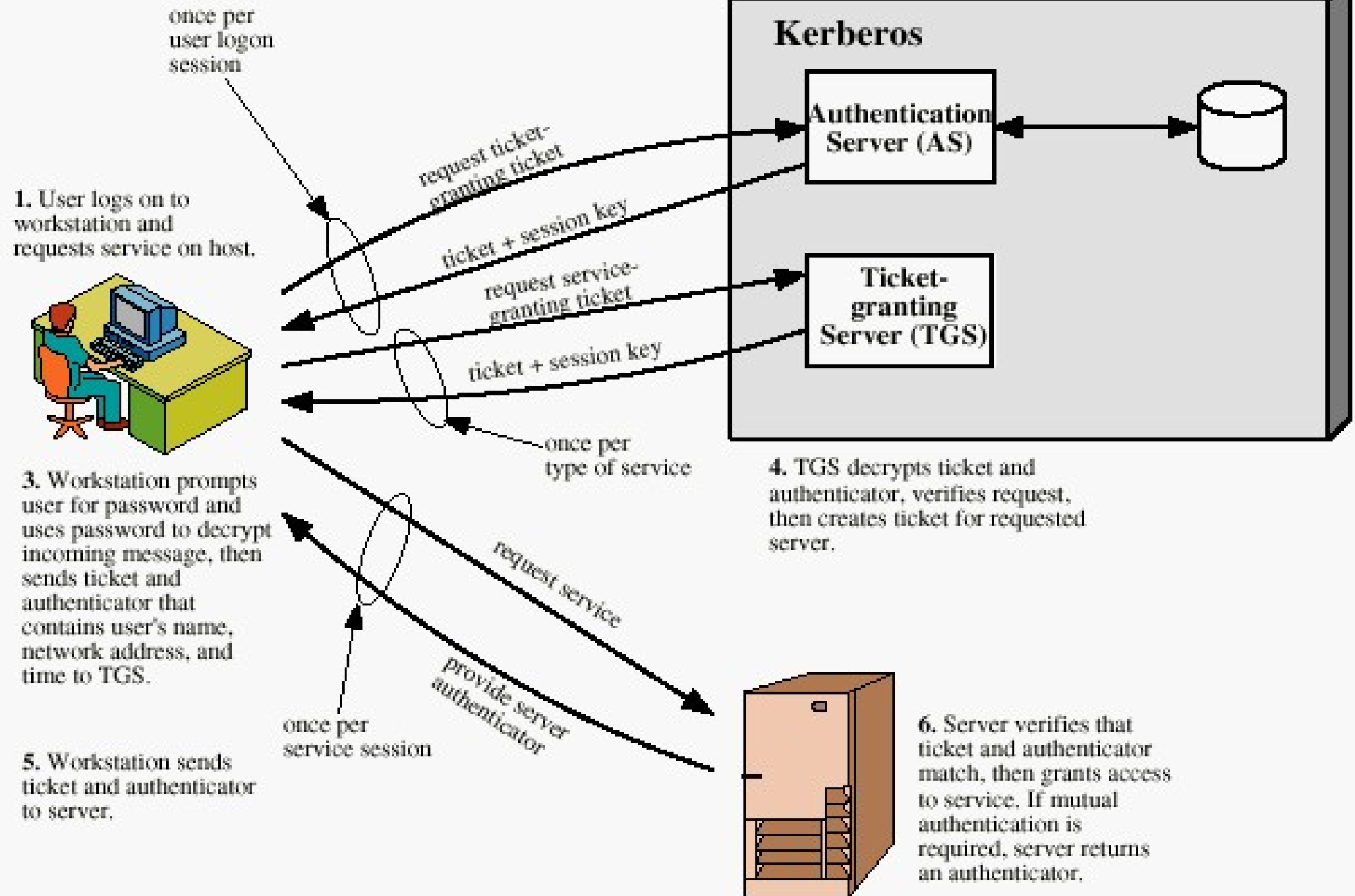
A session key $K_{c,\text{tgs}}$ is passed secretly to Client and TGS by AS.

Version 4 Authentication Dialogue

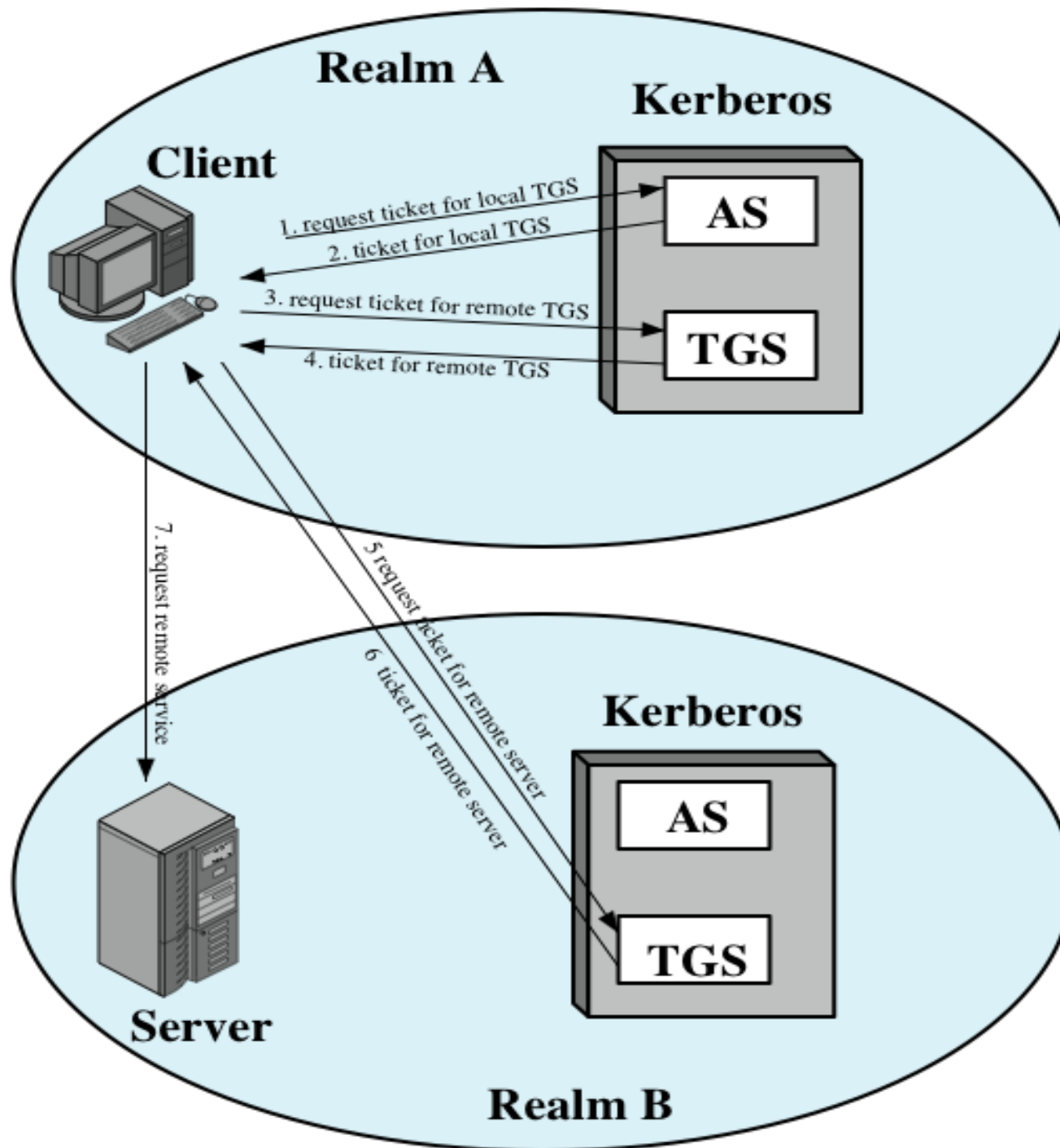
(a) Authentication Service Exchange: to obtain ticket-granting ticket
(1) $C \rightarrow AS: ID_c \parallel ID_{tgs} \parallel TS_1$
(2) $AS \rightarrow C: E_{K_c} [K_{c,tgs} \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2 \parallel Ticket_{tgs}]$ $Ticket_{tgs} = E_{K_{tgs}} [K_{c,tgs} \parallel ID_C \parallel AD_C \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2]$
(b) Ticket-Granting Service Exchange: to obtain service-granting ticket
(3) $C \rightarrow TGS: ID_v \parallel Ticket_{tgs} \parallel Authenticator_c$
(4) $TGS \rightarrow C: E_{K_{c,tgs}} [K_{c,v} \parallel ID_v \parallel TS_4 \parallel Ticket_v]$ $Ticket_{tgs} = E_{K_{tgs}} [K_{c,tgs} \parallel ID_C \parallel AD_C \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2]$ $Ticket_v = E_{K_v} [K_{c,v} \parallel ID_C \parallel AD_C \parallel ID_v \parallel TS_4 \parallel Lifetime_4]$ $Authenticator_c = E_{K_{tgs}} [ID_C \parallel AD_C \parallel TS_3]$
(c) Client/Server Authentication Exchange: to obtain service
(5) $C \rightarrow V: Ticket_v \parallel Authenticator_c$
(6) $V \rightarrow C: E_{K_{c,v}} [TS_5 + 1]$ (for mutual authentication) $Ticket_v = E_{K_v} [K_{c,v} \parallel ID_C \parallel AD_C \parallel ID_v \parallel TS_4 \parallel Lifetime_4]$ $Authenticator_c = E_{K_{c,v}} [ID_C \parallel AD_C \parallel TS_5]$

Overview of Kerberos

2. AS verifies user's access right in database, creates ticket-granting ticket and session key. Results are encrypted using key derived from user's password.



Request for Service in Another Realm



Message Exchange for Remote Realm Access

- 1) $C \rightarrow AS: ID_c || ID_{tgs} || TS_1$
- 2) $AS \rightarrow C: E_{K_c}[K_{c,tgs} || ID_{tgs} || TS_2 || Lifetime_2 || Ticket_{tgs}]$
- 3) $C \rightarrow TGS: ID_{tgsrem} || Ticket_{tgs} || Authenticator_c$
- 4) $TGS \rightarrow C: E_{K_{c,tgs}}[K_{c,tgsrem} || ID_{tgsrem} || TS_4 || Ticket_{tgsrem}]$
- 5) $C \rightarrow TGS_{rem}: ID_{vrem} || Ticket_{tgsrem} || Authenticator_c$
- 6) $TS \rightarrow C: E_{K_{c,tgsrem}}[K_{c,vrem} || ID_{vrem} || TS_6 || Ticket_{vrem}]$
- 7) $C \rightarrow V_{rem}: Ticket_{vrem} || Authenticator_c$

The only problem with this exchange is that when there are many realms

There are too many key exchanges to different realms.

Difference Between Version 4 and 5

A. Environmental Shortcomings

- Encryption system dependence (V.4 DES)
- Internet protocol dependence
- Message byte ordering
- Ticket lifetime
- Authentication forwarding
- Interrealm authentication

Differences Between V4 and V5 ...

B. Technical deficiencies

- **Double encryption:**
- **PCBC encryption:** Propagating CBC is used in V4 and this is non-standard. In V5 CBC is used.
- **Session key:** Each ticket includes a session key that can be used to encrypt the authenticator. Since the same key is used repeatedly to gain a service from particular server, there is a risk that an opponent can replay messages from an old session to the client or server. In V5 this is avoided by requiring a sub session key which is used only for one connection.

Version 5 Authentication Dialogue

(a) Authentication Service Exchange: to obtain ticket-granting ticket

(1) $C \rightarrow AS$: $Options \parallel ID_c \parallel Realm_c \parallel ID_{tgs} \parallel Times \parallel Nonce_1$

(2) $AS \rightarrow C$: $Realm_c \parallel ID_C \parallel Ticket_{tgs} \parallel E_{K_c} [K_{c,tgs} \parallel Times \parallel Nonce_1 \parallel Realm_{tgs} \parallel ID_{tgs}]$

$$Ticket_{tgs} = E_{K_{tgs}} [Flags \parallel K_{c,tgs} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times]$$

(b) Ticket-Granting Service Exchange: to obtain service-granting ticket

(3) $C \rightarrow TGS$: $Options \parallel ID_v \parallel Times \parallel Nonce_2 \parallel Ticket_{tgs} \parallel Authenticator_c$

(4) $TGS \rightarrow C$: $Realm_c \parallel ID_C \parallel Ticket_v \parallel E_{K_{c,tgs}} [K_{c,v} \parallel Times \parallel Nonce_2 \parallel Realm_v \parallel ID_v]$

$$Ticket_{tgs} = E_{K_{tgs}} [Flags \parallel K_{c,tgs} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times]$$

$$Ticket_v = E_{K_v} [Flags \parallel K_{c,v} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times]$$

$$Authenticator_c = E_{K_{c,tgs}} [ID_C \parallel Realm_c \parallel TS_1]$$

(c) Client/Server Authentication Exchange: to obtain service

(5) $C \rightarrow V$: $Options \parallel Ticket_v \parallel Authenticator_c$

(6) $V \rightarrow C$: $E_{K_{c,v}} [TS_2 \parallel Subkey \parallel Seq\#]$

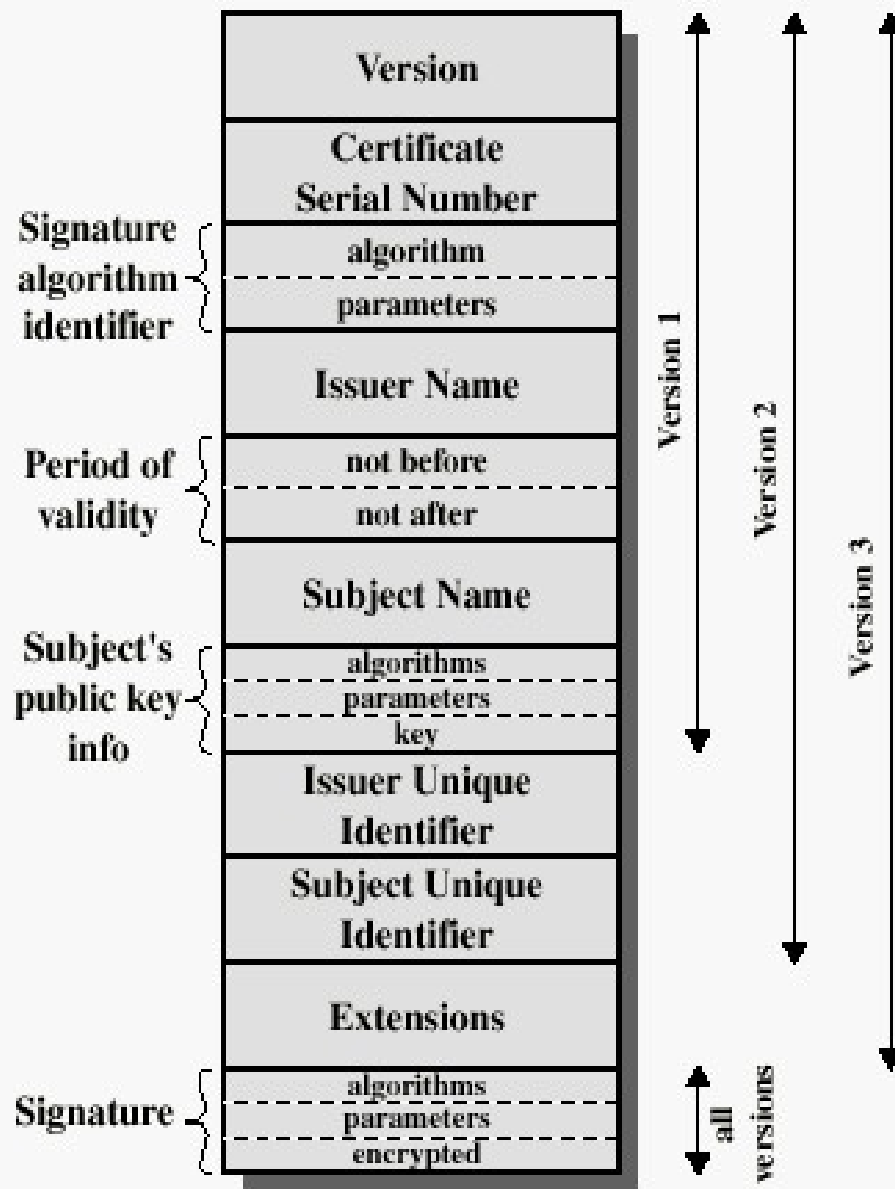
$$Ticket_v = E_{K_v} [Flags \parallel K_{c,v} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times]$$

$$Authenticator_c = E_{K_{c,v}} [ID_C \parallel Realm_c \parallel TS_2 \parallel Subkey \parallel Seq\#]$$

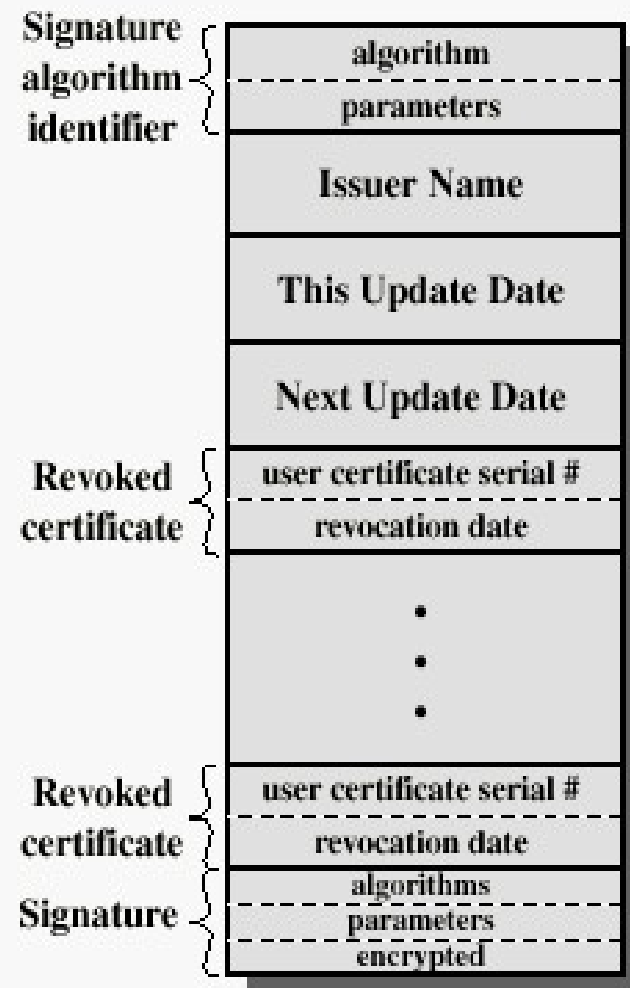
X.509 Authentication Service

- More broadly X.509 defines a framework to provide a secure directory services.
- Distributed set of servers that maintains a database about users.
- Each certificate contains the public key of a user and is signed with the private key of a CA.
- Is used in S/MIME, IP Security, SSL/TLS and SET.
- RSA is recommended to use

X.509 Formats



(a) X.509 Certificate



(b) Certificate Revocation List

X.509 Certificate Example

Certificate:

Data:

Version: 1 (0x0)

Serial Number: 7829 (0x1e95)

Signature Algorithm: md5WithRSAEncryption

Issuer: C=ZA, ST=Western Cape, L=Cape Town, O=Thawte Consulting cc, OU=Certification Services Division,
CN=Thawte Server CA/emailAddress=server-certs@thawte.com

Validity

Not Before: Jul 9 16:04:02 1998 GMT

Not After : Jul 9 16:04:02 1999 GMT

Subject: C=US, ST=Maryland, L=Pasadena, O=Brent Baccala,
OU=FreeSoft, CN=www.freesoft.org/emailAddress=baccala@freesoft.org

Subject Public Key Info:

Public Key Algorithm: rsaEncryption

RSA Public Key: (1024 bit)

Modulus (1024 bit):

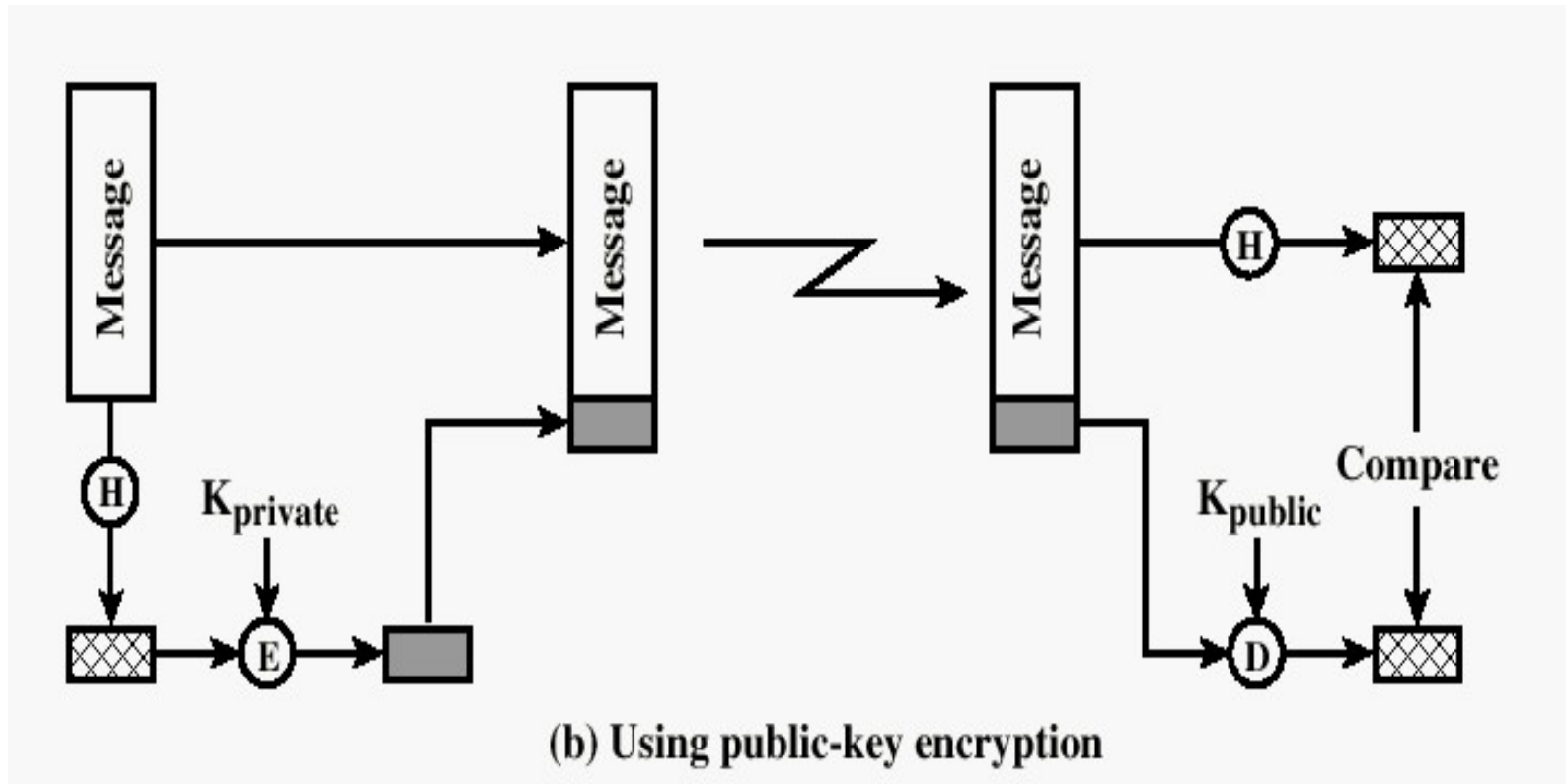
00:b4:31:98:0a:c4:bc:62:c1:88:aa:dc:b0:c8:bb:33:35:19:d5:0c:64:b9:3d:41:b2:96:fc:f3:31:e1:
66:36:d0:8e:56:12:44:ba:75:eb:e8:1c:9c:5b:66:70:33:52:14:c9:ec:4f:91:51:70:39:de:53:85:17:
16:94:6e:ee:f4:d5:6f:d5:ca:b3:47:5e:1b:0c:7b:c5:cc:2b:6b:c1:90:c3:16:31:0d:bf:7a:c7:47:77:
8fa0:21:c7:4c:d0:16:65:00:c1:0f:d7:b8:80:e3:d2:75:6b:c1:ea:9e:5c:5c:ea:7d:c1:a1:10:bc:b8:
e8:35:1c:9e:27:52:7e:41:8f

Exponent: 65537 (0x10001)

Signature Algorithm: md5WithRSAEncryption

93:5f:8f:5f:c5:af:bf:0a:ab:a5:6d:fb:24:5f:b6:59:5d:9d:92:2e:4a:1b:8b:ac:7d:99:17:5d:cd:19:f6:ad:ef:63:2f:92:
ab:2f:4b:cf:0a:13:90:ee:2c:0e:43:03:be:f6:ea:8e:9c:67:d0:a2:40:03:f7:ef:6a:15:09:79:a9:46:ed:b7:16:1b:41:72:
0d:19:aa:ad:dd:9a:df:ab:97:50:65:f5:5e:85:a6:ef:19:d1:5a:de:9d:ea:63:cd:cb:cc:6d:5d:01:85:b5:6d:c8:f3:d9:f7:
8f:0e:fc:ba:1f:34:e9:96:6e:6c:cf:f2:ef:9b:bf:de:b5:22:68:9f

Typical Digital Signature Approach



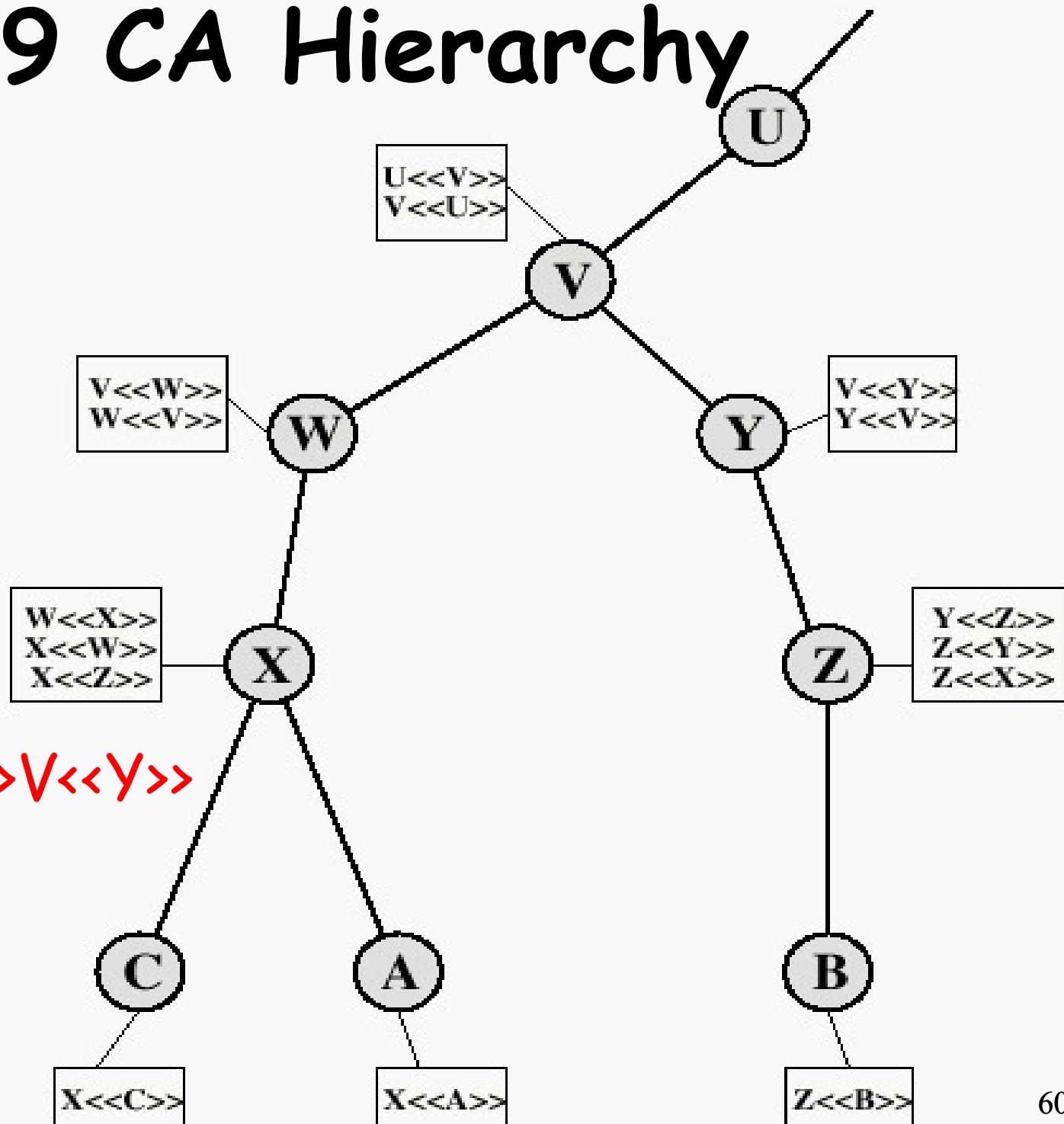
Obtaining a User's Certificate

- Characteristics of certificates generated by CA:
 - Any user with access to the public key of the CA can verify the user public key that was certified.
 - No party other than the CA can modify the certificate without this being detected.

X.509 CA Hierarchy

User A acquires following certificates from the directory to establish certification path to user B:

$X \ll W \gg W \ll V \gg V \ll Y \gg$
 $Y \ll Z \gg Z \ll B \gg$



Authentication Procedures

X.509 includes three alternative authentication procedures:

1. One-Way Authentication
 2. Two-Way Authentication
 3. Three-Way Authentication
- all use public-key signatures

1. One-Way Authentication

- 1 message ($A \rightarrow B$) used to establish
 - the identity of A and that message is from A
 - message was intended for B
 - integrity & originality of message
- message must include timestamp, nonce, B's identity and is signed by A

Authentication Procedures

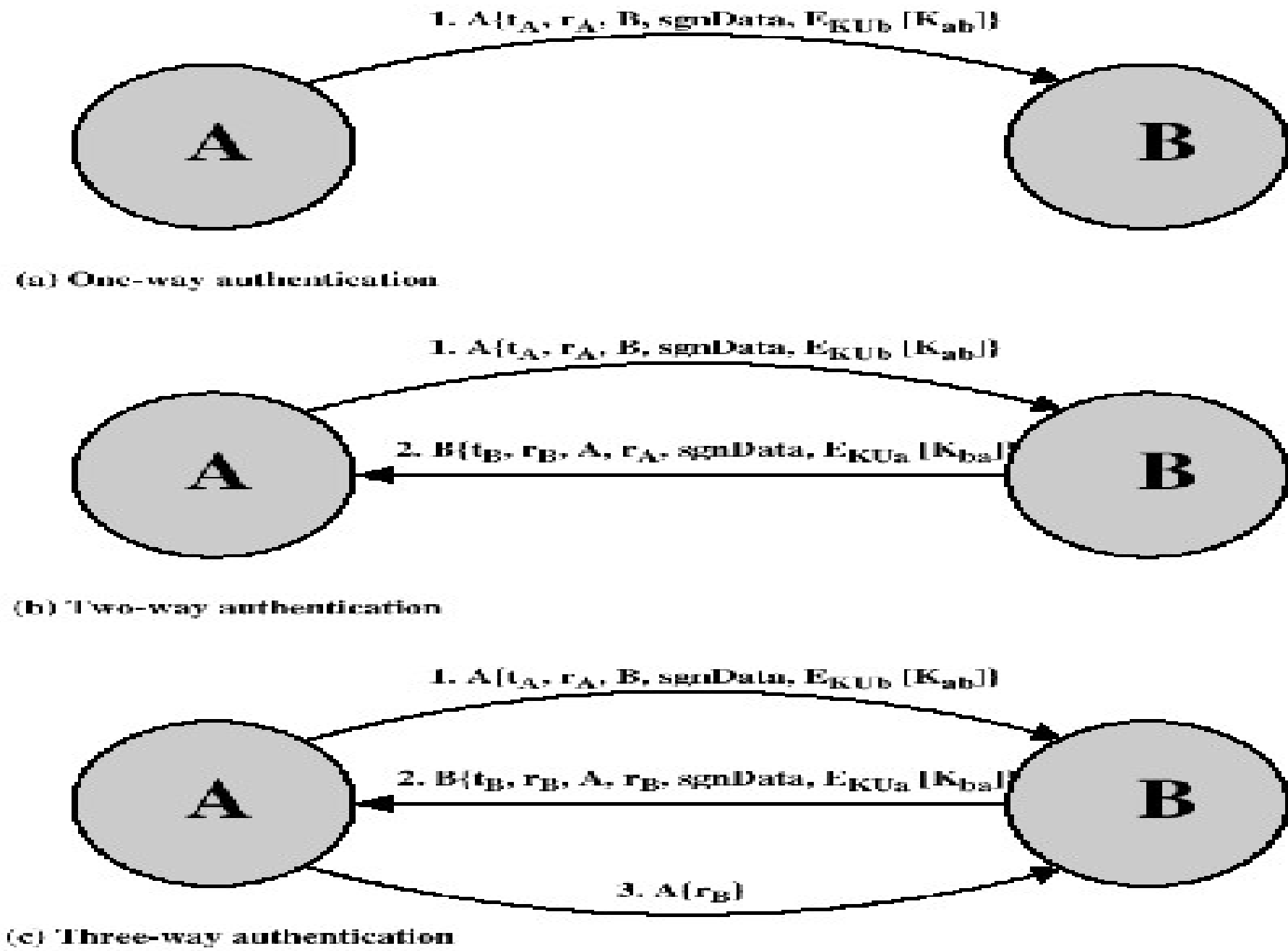


Figure 4.5 X.509 Strong Authentication Procedures

2. Two-Way Authentication

- 2 messages ($A \rightarrow B$, $B \rightarrow A$) which also establishes in addition:
 - the identity of B and that reply is from B
 - that reply is intended for A
 - integrity & originality of reply
- reply includes original nonce from A, also timestamp and nonce from B

3. Three-Way Authentication

- 3 messages ($A \rightarrow B$, $B \rightarrow A$, $A \rightarrow B$) which enables above authentication without synchronized clocks
- has reply from A back to B containing signed copy of nonce from B
- means that timestamps need not be checked or relied upon

Description of X.509 Version 3

The following requirements not satisfied by version 2:-

1. Subject field is inadequate to convey the identity of a key owner.
2. The subject field is also inadequate for many applications
3. There is need to indicate security policy information
4. There is a need to specify particular public key among several public keys of same CA or user
5. Need to specify constraints on the applicability of a particular certificate

- Version 3 includes "**extension**" field, which is optional. The certificate extensions fall into three main categories.
 - (1) Key and policy information
 - (2) Subject and issuer attributes
 - (3) Certification path constraints
- (1) Key and policy information:
 - i) Authority key identifier
 - ii) Subject key identifier
 - iii) key usage
 - iv) private key usage period
 - v) certificate policies
 - vi) policy mappings

- (2) Subject and Issuer attributes:
 - (I) Subject alternative name
 - (ii) Issuer alternative name
 - (iii) Subject directory attribute

- (3) Certification path constraints:
 - (I) Basic constraints: If subject is CA, then certification path length constraint may be specified.
 - (ii) Name constraints
 - (iii) Policy constraints: Inhibit policy mapping for the remainder of the certification path

Revocation of Certificates

- Reasons for revocation:
 1. The users secret key is assumed to be compromised.
 2. The user is no longer certified by this *CA*.
 3. The *CA*'s certificate is assumed to be compromised.